

How F2Z performs the forest GKR: the merged, lazy, bit-affine grand-product forest

The `f2z-pcs` repository

2026-07-31

Abstract

F2Z’s integer-MLE opening rides on a grand-product argument: per column c of the committed bit matrix, the prover must certify $\alpha^{u_c} = \prod_i \Lambda[c][i]$ for the *bit-affine* leaves $\Lambda[c][i] = 1 + M[c][i] \cdot \tau(i)$. This is the prover’s dominant phase (81–96% of prove time across $n = 16\text{--}32$), and this note explains how it is actually performed, as implemented in `src/merged_forest.rs` and the eq-factored sumcheck driver it runs on. Three ideas carry the design. (i) *Merging*: the 2^s product trees are one multilinear family with the tree index as extra MLE variables, so one GKR run binds every root and each layer’s payload is $O(\ell + s)$ field elements instead of $O(2^s)$. (ii) *The two-phase eq-factored layer sumcheck*: phase A binds the in-tree variables with the trees as eq-weighted groups sharing one reduction point; phase B binds the tree-index variables over the per-tree finals, which never enter the transcript. The driver materialises no `eq` tables, accumulates round coefficients in unreduced 256-bit form, and gets the fourth Lagrange node for free from a characteristic-2 affine-flat identity. (iii) *Laziness*: the leaf layer and the bottom product levels are never built — the bottom layers run branchless case-LUT sumcheck rounds straight off the packed committed bits against shared τ -product tables, one intermediate level is regenerated just in time (with its first round message fused into the generation pass), and the stored chain tops out a quarter (default schedule) or an eighth (18) of the way up the tree, which is what caps prover memory at $\approx 2^n \cdot 4\text{ B}$ resp. $\approx 2^n \cdot 2\text{ B}$. Every lazy path is an exact characteristic-2 rewriting of the eager forest: the transcripts are byte-identical, and tests pin this in every schedule. The note closes with the exit-claim discharge into the ring-switch/Ligerito opening, the batched and RLC-family variants of the forest, and the measured cost tables.

Contents

1	The job of the forest	2
2	One forest, one transcript: the merged statement	3
3	The layer protocol	4
4	The eq-factored driver	5
5	Leaves from bits: the lazy schedules	6
5.1	Two square laws and one cap	6
5.2	What the prover holds	7
5.3	The level ladder	9
5.4	Inside the case-LUT rounds	10
5.5	Why byte-identity holds, and how it is pinned	13

6	Memory and cost	14
7	The exit claim and its discharge	15
8	Variants: the batched forest and the RLC families	15
9	What to trust, and what this note is not	16

1 The job of the forest

F2Z¹ proves an *integer* multilinear-extension evaluation $\widetilde{\text{INT}}(D)(r) = y \in \mathbb{F}_q$ over data D whose *bits* are committed with a characteristic-2 commitment. The instance shape is (t, s, W) (`IntEvalParams`): D is a $2^t \times 2^s$ matrix of W -bit cells, its bits arranged as the single-bit matrix

$$M[c][i] \in \{0, 1\}, \quad i = (b \ll \log_2 W) | j \in \{0, 1\}^d, \quad d = t + \log_2 W,$$

with b the row, c the column, j the bit position inside the cell. The t row variables are folded with integer weights, the s column variables are read off in the clear. We write $n = t + s$ for the benchmark-scale parameter (the deployed shapes have $W = 1$, so 2^n is the committed bit count) and $\mathbb{K} = \text{GF}(2^{128})$ for the binding field, with $\alpha \in \mathbb{K}^\times$ a generator.

The claim the forest certifies. An integer fold cannot be read through the $\mathbb{F}_2$ commitment (parities destroy magnitudes), so F2Z carries it in the exponent of $\mathbb{K}$: for one chunk l of the mod- q opening, with public c_w -bit chunk weights $W_b^{(l)}$ (limbs of the $\mathbb{F}_q$ row-eq weights, $c_w = 127 - t - W$), the prover sends the per-column integer folds

$$u_c^{(l)} = \sum_b W_b^{(l)} \cdot \text{INT}(D(b, c)) < 2^{c_w+t+W} = 2^{127},$$

and must certify them against the commitment. The certificate is a grand product: each factor is *bit-affine* in one committed bit,

$$\Lambda^{(l)}[c][i] = 1 + M[c][i] \cdot \tau^{(l)}(i), \quad \tau^{(l)}(i) = \alpha^{W_b^{(l)} 2^j} - 1 \quad (i = (b \ll \log_2 W) | j), \quad (1)$$

so that $\prod_i \Lambda^{(l)}[c][i] = \alpha^{u_c^{(l)}}$. (In $\mathbb{K}$, $-1 = +1$; we keep the -1 form for readability.) The forest's output is a reduction of all 2^s product claims to *one* evaluation claim on the leaf MLE $\tilde{\Lambda}$, which §7 converts into a $\mathbb{K}$ -linear functional of the committed bits and hands to the ring-switch + Ligerito opener. Range checks on the sent u 's and the generator property of α make the exponent map injective on the values that occur, so the products bind the integers; that chain of checks is the verifier note's §§5–7, and we do not repeat it here.

Why performance of this phase is the whole game. Time is forest-dominated: the **forest+presum** phase is 81 % of prove at $n = 16$, rising to 92–96 % at $n \geq 26$ (§6). Proof *bytes*, by contrast, are dominated by the Ligerito opening — the forest side stays at 8–51 KiB through $n = 28$. So the forest is engineered for prover time and memory, and its transcript is engineered to stay small; both axes are visible in the design below.

¹Repository `f2z-pcs`, a standalone extraction of the `f2-int` pipeline from `zinc-plus` (branch `f2-int-unified-ligerito`); the construction is due to Lev Soukhanov (the `char2-fieldswitch` note, §9). `docs/DESIGN.md` holds the distilled protocol; `docs/verifier-note/` specifies every verifier check; `docs/rlc-family-note/` develops the experimental claim families. This note is the *prover-side* companion: how the forest is computed, scheduled, and kept small.

Scope. Everything in this note is the code of this repository: the merged forest (`src/merged_forest.rs`), the eq-factored driver (`src/piop/sumcheck/eq_factored.rs`), and the table builders (`src/pcs.rs`). Function names are given so claims can be checked against source; line numbers are avoided (the files drift). The Ligerito opener is out of scope (black box, as in the verifier note).

2 One forest, one transcript: the merged statement

A naive realisation runs one GKR product tree per column and carries, per tree per layer, a pair of child evaluations — $2 \cdot 2^s \cdot d$ $\mathbb{K}$ -elements of proof, ≈ 512 KiB at $n = 26$. The merged forest removes the 2^s factor from the payload by treating the tree index as *more MLE variables*.

The layer family. Index the values of tree c at depth ℓ (the root is depth 0, leaves depth d) by $x \in \{0, 1\}^\ell$, and define one multilinear family per depth,

$$L_\ell(x, c), \quad (x, c) \in \{0, 1\}^\ell \times \{0, 1\}^s,$$

with x occupying the *low* index bits and c the high ones. The product recursion pairs children on the *top* in-tree bit:

$$L_\ell(x, c) = L_{\ell+1}(x, 0, c) \cdot L_{\ell+1}(x, 1, c), \quad (2)$$

i.e. the two children of node x sit at positions x and $x + 2^\ell$ of level $\ell + 1$. This convention is load-bearing for the lazy schedules: every level is stored and consumed as a top-bit split pair of halves (E, O) — $E_c(x) = L_{\ell+1}(x, 0, c)$, $O_c(x) = L_{\ell+1}(x, 1, c)$ — and the leaf halves are exactly “first 2^{d-1} bits, last 2^{d-1} bits” of a column, which is what lets pair products be table lookups (§5). $L_0(\cdot, c) = R_c$ are the roots and $L_d = \Lambda$ the leaves.

Entry: the roots are derived, absorbed, and evaluated — never sent per-tree. The prover’s product-tree build produces the roots R_c as a byproduct; the proof object carries no root field. On the verifier’s side the roots are *recomputed* as $R_c = \alpha^{u_c}$ from the sent folds (fixed-base comb, `FixedBasePow`) — for an honest prover these are the same values, and that equality is precisely the claim being proven. Both sides then perform the same two transcript actions (`drive_grouped` / `verify_merged_forest`):

$$\text{absorb}(0x30 : R_0, \dots, R_{2^s-1}), \quad \zeta \leftarrow \text{squeeze}(\mathbb{K}^s),$$

and each computes the entry claim itself, $C_0 = \tilde{R}(\zeta)$ (`mle_at`, $O(2^s)$ work). The absorb is the binding: a prover whose subsequent messages were generated against different roots faces challenges derived from *these* roots. Note the prover-side integer folds u_c (`fold_values_bits`, a popcount-style pass over the packed rows) are computed independently of the forest and enter the proof stream as the `us` vector; the forest itself never touches them.

Per-layer payload. Each of the d layers contributes one `MergedLayer` to the proof: an optional phase-A sumcheck (ℓ rounds, absent at $\ell = 0$), a phase-B sumcheck (s rounds), and a closing child pair — $O(\ell + s)$ $\mathbb{K}$ -elements. Summed over layers this is $\approx 48 \left(\frac{d(d-1)}{2} + ds \right)$ bytes of round messages plus $32d$ bytes of pairs (degree-3 rounds carry three 16-byte tail values each, §4); at the deployed shapes the *sent folds* $16 \cdot 2^s$ B are the larger forest-side item (e.g. $n = 28$, $s = 11$: 32 KiB of folds vs. ≈ 16 KiB of forest messages, 50.7 KiB forest-side total).

3 The layer protocol

The verifier maintains a running claim $C_\ell = \widetilde{L}_\ell(z_x, z_c)$ starting from $(z_x, z_c) = (\varepsilon, \zeta)$. By (2) and multilinearity, the layer- ℓ claim factors coordinate-wise:

$$\begin{aligned} C_\ell &= \sum_{x \in \{0,1\}^\ell} \sum_{c \in \{0,1\}^s} \text{eq}(x, z_x) \text{eq}(c, z_c) \underbrace{L_{\ell+1}(x, 0, c)}_{E_c(x)} \underbrace{L_{\ell+1}(x, 1, c)}_{O_c(x)} \\ &= \sum_c \text{eq}(c, z_c) \left[\sum_x \text{eq}(x, z_x) E_c(x) O_c(x) \right]. \end{aligned} \quad (3)$$

The bracketed inner sums share the point z_x and differ only in the tree; that shape is exactly one *shared- q multi-group* instance of the eq-factored driver (§4), and the layer is discharged in two phases plus a line step (**drive_grouped**):

1. **Phase A** (in-tree variables; $\ell \geq 1$ only). The 2^s trees enter as groups: group c has the shared eq point $q = z_x$, the per-group scale $\text{eq}(c, z_c)$ (a precomputed $O(2^s)$ table re-built per layer), and the single buffer pair (E_c, O_c) . One driver call (**prove_eq_inner_sumcheck_mixed_pre**) runs ℓ rounds of a degree-3 sumcheck binding $x \rightarrow r_x \in \mathbb{K}^\ell$, and returns the per-tree finals $(\widetilde{E}_c(r_x), \widetilde{O}_c(r_x))$. These finals are *prover-internal*: nothing per-tree is absorbed at any point.
2. **Phase B** (tree-index variables). The residual $\sum_c \text{eq}(c, z_c) \widetilde{E}_c(r_x) \widetilde{O}_c(r_x)$ is itself an eq-weighted inner product over c — the per-tree finals, read as two length- 2^s multilinear tables $E(\cdot), O(\cdot)$, feed one more (tiny, dense) driver call with $q = z_c$, binding $c \rightarrow r_c \in \mathbb{K}^s$ and closing on the child pair

$$(p, q) = (\widetilde{L}_{\ell+1}(r_x, 0, r_c), \widetilde{L}_{\ell+1}(r_x, 1, r_c)).$$

3. **Line step. absorb(0x32 : p, q), $\lambda_\ell \leftarrow \text{squeeze}(\mathbb{K})$** , and the claim advances along the line through the two children:

$$C_{\ell+1} = p + \lambda_\ell (p + q), \quad z_x \leftarrow r_x \parallel \lambda_\ell, \quad z_c \leftarrow r_c.$$

(The code calls λ_ℓ **mu**; renamed to keep μ for the discharge residual, as in the verifier note.)

Layer 0 degenerates: there are no in-tree variables yet, phase A is skipped (**sc_x = None**), and the pair vectors fed to phase B are simply each root's two children — level 1, two values per tree. After d layers the *exit claim* is

$$e_d = C_d \stackrel{\text{honest}}{=} \widetilde{\Lambda}(z_x, z_c), \quad z = (z_x, z_c) \in \mathbb{K}^{d+s},$$

pinned by the **merged_forest_roundtrips** test against **mle_at(leaves, z)**. The verifier-side chaining checks (phase-A claimed sum $= C_\ell$, the factored-eq linkage $E_A = \text{eq}(r_x, z_x) \cdot \text{sc}_c.\text{claimed_sum}$, the closing check $E_B = \text{eq}(r_c, z_c) \cdot p \cdot q$) are [V5]–[V11] of the verifier note and mirror this structure exactly.

Remark 1 (Why two phases). A single $(\ell + s)$ -variable sumcheck over the merged index would be sound too, but would need the $\text{eq}(c, z_c)$ weights woven into every round over a buffer of size $2^{\ell+s}$. The split lets phase A treat $\text{eq}(c, z_c)$ as a per-group scalar (it seeds the group's prefix scalar, §4, at zero marginal cost), keeps the per-tree buffers independent — which is what the lazy bit-driven rounds and the across-tree parallelism need — and makes phase B negligible (2^s -sized buffers). The transcript cost is one extra sumcheck header per layer.

4 The eq-factored driver

Every phase of every layer runs on one prover, `prove_eq_inner_sumcheck_mixed_pre` (`src/piop/sumcheck/eq_factored.rs`), which proves sums of the shape

$$\sum_x \sum_g \text{eq}(x; q_g) \cdot \text{scale}_g \cdot L_g(x) R_g(x)$$

— an eq-weighted sum of inner products of multilinear pairs — and is **byte-identical** to the generic `MLSumcheck` prover run over the materialised comb $[\text{eq}_1, \dots, L\text{'s}, R\text{'s}]$ with the degree-3 combiner $\sum_g \text{eq}_g L_g R_g$: same round polynomials, same transcript operations (header, tail absorbs, post-draw challenge re-absorb), same proof layout. The generic verifier verifies it unchanged — there is no bespoke verification path to audit.

Never materialise eq. Round j (binding the *low* variable, as the generic prover does) writes each group’s round message as

$$P_j(c) = \sum_g A_{g,j} \text{eq}_1(c; q_g[j-1]) H_{g,j}(c), \quad H_{g,j}(c) = \sum_b V_{g,j}[b] L_g(c, b) R_g(c, b),$$

where $L_g(c, b) = L_g[2b] + c(L_g[2b+1] - L_g[2b])$ is the fold-in-progress buffer at suffix slot b , and the eq weight has been split into three parts, none of them a materialised multiplicand:

- the *prefix scalar* $A_{g,j} = \text{scale}_g \cdot \prod_{i < j-1} \text{eq}_1(\rho_{i+1}; q_g[i])$, one running $\mathbb{K}$ -element per group, updated per round — this is where phase A’s $\text{eq}(c, z_c)$ group scale rides for free;
- the current-variable factor $\text{eq}_1(c; q_g[j-1])$, degree 1 in the round variable;
- the *suffix tensor* $V_{g,j}[b] = \prod_{i \geq j} \text{eq}_1(b_{i-j}; q_g[i])$, precomputed back-to-front for all rounds in $O(2^k)$ total (`suffix_tensors`) — the cost of one eq build, and built *once* for the whole phase-A call, since all trees share $q = z_x$ (asserted: bit-selected groups require the shared point).

H is quadratic and the eq_1 factor linear, so P_j has degree 3, matching the pinned verifier degree.

Coefficient-space accumulation, wide sums, free fourth node. The per-slot inner loop accumulates the *coefficients* (A_0, A_1, A_2) of H in the round variable rather than its node values, so the Lagrange-node multiplies stay out of the hot loop; the single-pair case costs 5 field multiplies per slot (the suffix weight is folded into L : $w \cdot l_0$, $w \cdot l_1$, then three products $(wl_0)r_0$, $(wl_1)r_1$, $\Delta L \cdot \Delta R$). The three products feed `WideMulAcc` accumulators: unreduced 256-bit carry-less sums with *one* polynomial reduction per accumulator per round — exact, because the reduction is $\mathbb{F}_2$ -linear. Round messages are sent as tail evaluations $P_j(1), P_j(2), P_j(3)$ only; the verifier reconstructs $P_j(0)$ from the chained sum (the claimed-sum trick [4]). The fourth node is free in characteristic 2: the evaluation points $\{0, 1, X, X+1\}$ (bit-pattern convention) form an $\mathbb{F}_2$ -affine 2-flat, and over such a flat every polynomial of degree ≤ 2 sums to zero, so $H(X+1) = H(0) + H(1) + H(X)$. The detection is exact at runtime ($1 + 1 = 0$), keeping the driver field-generic. On aarch64 the whole-buffer message and fold passes dispatch to fused NEON kernels (`eqf_single_pair_round` and friends; `F2Z_EQF_NOKERNEL=1` forces the generic loops for A/B diagnosis).

Pass fusion. By default the driver defers each round’s fold and runs it *fused into the next round’s message pass*: one read of the unfolded buffers replaces a fold-write pass plus a message-read pass. Measured $\approx 10\%$ of prove at $n = 26\text{--}28$; byte-identical (same field values in the same transcript order — only the physical pass structure changes); `F2Z_EQF_FUSE=0` restores the eager two-pass path. The same idea appears one level up in the forest’s JIT layers: the *buffer-generation* pass accumulates the round-1 coefficients as it writes (`dense_jit_fused_round1`, forwarded through the driver’s `pre_round1` hook), so a JIT-generated level is written once and first *read* by round 2’s fused fold+message pass (`F2Z_JIT_R1=0` opts out). XOR accumulation is order-free and reduction $\mathbb{F}_2$ -linear, hence generation-order independence of the reduced coefficients — the fusion is again an exact identity.

5 Leaves from bits: the lazy schedules

The eager reference prover (`prove_merged_forest`) materialises the 2^{d+s} leaves and every product level above them. At the deployed shapes that is not a constant factor away from affordable: at $n = 28$ ($d = 17$, $s = 11$) the leaf layer alone is 2^{28} $\mathbb{K}$ -elements = 4 GiB, the level above it 2 GiB, the whole forest ≈ 8 GiB — whereas the measured peak resident set of an *entire* $n = 28$ prove is 1.27 GB on the 16 GB M4 of §6. The lazy prover (`prove_merged_forest_lazy`) produces a byte-identical transcript while never materialising the leaf layer or the bottom product levels at all.

Everything rests on one observation: near the leaves the forest’s values carry almost no entropy. A node r levels above the leaves is the product of the 2^r leaves beneath it, and every leaf is either 1 or a public table entry (1) — so the node’s value is one of 2^{2^r} possibilities, *selected by 2^r committed bits*, out of a table that depends on the node’s position but not on its tree. For small r that replaces “compute and store 2^s copies of the level” by “gather from one shared table indexed by committed bits”. §5.1 turns the observation into the two inequalities that generate every variant in the code, §5.2 lists what the prover actually holds, §5.3 assigns every level of the tree to a sourcing mode, §5.4 works through the round bodies, and §5.5 says why the result is the same protocol.

One index convention first, because it ties the two halves of this section together: layer ℓ ’s phase A binds the ℓ in-tree variables over level $\ell + 1$, which has 2^ℓ values per half — so *layer ℓ is a $k = \ell$ variable instance of the driver*, and the bottom three layers of a depth- d tree run at $k = d - 3, d - 2, d - 1$. Every “ $k \geq \cdot$ ” guard below is a statement about depth.

5.1 Two square laws and one cap

Two independent operations widen the case tables the lazy prover reads, and both *square* the number of cases. The shape of the whole prover follows from where their common cap bites.

Identity 1 (Level width). Unfolding the recursion (2) r times, position y of level $d - r$ in tree c is

$$L_{d-r}(y, c) = \prod_{\varepsilon \in \{0,1\}^r} \Lambda[c] \left[y + \sum_{i=1}^r \varepsilon_i 2^{d-i} \right],$$

the product over the 2^r leaves whose low $d - r$ index bits are y (the top r bits run free). With $\Lambda[c][i] \in \{1, v(i)\}$, $v(i) = \alpha^{W_b 2^i}$, that value is one of 2^{2^r} position-indexed constants, selected by the 2^r committed bits of the subcube — and the constants do not depend on c .

A level- $(d-r)$ case table therefore holds $2^{2^r} \cdot 2^{d-r}$ entries: $2^{d+1}, 2^{d+1}, 2^{d+2}, 2^{d+5}$ for $r = 0, 1, 2, 3$, i.e. 4 MiB, 4 MiB, 8 MiB, **128 MiB** at $d = 17$. The first three are built once, shared across all $2^8 = 2048$ trees, and stay cache-resident; the fourth does not exist in the code, and that 16-fold jump is why.

Identity 2 (Fold width). A sumcheck round replaces the entry pair (v_{2p}, v_{2p+1}) by $v'_p = v_{2p} + \rho(v_{2p+1} - v_{2p})$. If each v was a 2^w -case function of w bits at its own position, then v'_p is a 2^{2w} -case function of the $2w$ bits of the two positions — again a per-position table (now ρ -dependent), again shared by all trees.

So an entry of level $d - r$ with f variables already bound is a $2^{2^{r+f}}$ -case select on 2^{r+f} bits, and the same “at most 4 selecting bits” cap reads $r + f \leq 2$. A layer whose input level is $d - r$ can therefore run $3 - r$ rounds straight off the bits; and a prefix of f such rounds needs $k \geq f + 1$ variables, because the last of them must still have a fold — that fold is where the dense buffers are born. Those two inequalities generate the driver’s entire **GroupBufs** inventory, its assertions, and the depth fallbacks of §5.3:

input level	r	rounds	widest select	driver entry	needs
d (leaves)	0	1	1 bit / 2 cases	LeafBits	$k \geq 2$
		2	2 bits / 4	Leaf2Bits	$k \geq 3$
		3	4 bits / 16	Leaf3Bits	$k \geq 4$
$d - 1$	1	1	2 bits / 4	Pair2Bits	$k \geq 2$
		2	4 bits / 16	Pair3Bits	$k \geq 3$
$d - 2$	2	1	4 bits / 16	T4Bits	$k \geq 2$
$d - 3$	3	0	8 bits / 256	— (128 MiB)	—

5.2 What the prover holds

Bits, transposed once. The commitment hint stores the bit matrix column-lane-packed: word i of group g carries bit i of the 64 columns $64g \dots 64g+63$. The forest wants the opposite — one contiguous bit array per tree — so a single pass (**extract_column_bit_halves**) changes the layout with 64×64 in-register bit transposes (**transpose64** of 64 consecutive position-words yields, in lane j , the position-packed word of column $64g+j$) and writes each tree’s bits already split at the midpoint:

$$\text{lbits}_c[i] = M[c][i], \quad \text{rbits}_c[i] = M[c][i + 2^{d-1}], \quad i < 2^{d-1},$$

packed 64 per **u64** — $2^d/8$ bytes per tree, $2^n/8 = 32$ MiB for the whole matrix at $n = 28$. These two arrays drive *every* lazy layer of tree c ’s bottom. The split is not a convenience: by the top-bit convention of §2, the free index bits of Identity 1 are the *high* ones, so the top leaf-index bit of a subcube selects the array and the remaining $r - 1$ free bits are position offsets *inside* it. Concretely, a level- $(d-1)$ node at position i multiplies leaves i and $i + 2^{d-1}$, and its two selecting bits are bit i of **lbits** and bit i of **rbits** — the same index into both arrays; a level- $(d-2)$ node at position y takes bits y and $y + 2^{d-2}$ of both. Sweeping bits word-wise therefore costs 4 **u64** loads per 64 values, $\approx 64 \times$ less read traffic than per-bit **packed_col_bit** gathers over the lane-packed store, which touch a whole word per bit.

α -power chains. Per chunk, **chunk_pow2_table** builds $v(i) = \alpha^{W_b^{(i)} 2^j}$ for all $i = (b \ll \log_2 W)|j$. All 2^t rows share one α -squaring chain, so the row bases α^{W_b} come from a single fixed-base comb (**FixedBasePow**, window 8 — $\approx 8 \times$ fewer multiplies than per-row square-and-multiply), and a row’s

W place values are then its own squaring chain $\alpha^{w2^j} = (\alpha^{w2^{j-1}})^2$. Every table below is built from this one array.

The four shared table families. Built once per weight set (“ τ set”) and read by all 2^s trees. Write $q_1 = 2^{d-2}$, $q_2 = 2^{d-1}$.

- *leaf τ halves* (**leaf_tau_halves**): $\tau(i) = v(i) + 1$, split like the bits. This is the $r = 0$ table in minimal form — case 0 is the constant 1 and needs no storage, so one entry per position suffices and the leaf is recovered as the bit-affine $1 + m\tau$ of (1). (Padded rows have $v = \alpha^0 = 1$, hence $\tau = 0$: their leaves are 1 whatever bits they carry.)
- the *pair table* (**layer1_pair_table**): **pair** $[i] = v(i)v(i + q_2)$ for $i < q_2$ — the both-bits-set case of a level- $(d-1)$ node, and the only one of its four cases that is not already an entry of v . Building it once instead of once per column saves, at random bit density, a quarter of all pairs times 2^s columns of redundant $\mathbb{K}$ -multiplies.
- the $r = 1$ *case tables* **te**, **to**: for $y < q_1$,

$$\mathbf{te}[4y + c] = (1, v(y), v(y+q_2), \mathbf{pair}[y])_c, \quad \mathbf{to}[4y + c] = \text{the same at } y + q_1,$$

with case index $c = m_{\text{lo}} | m_{\text{hi}} \ll 1$ (lo = the first-half leaf, hi = the second-half one). **te** covers the E half of level $d-1$ (position y , leaves $y, y+q_2$), **to** the O half (position $y+q_1$, leaves $y+q_1, y+q_1+q_2$) — the offset is baked into the table, so both are read at the same index y .

- the $r = 2$ *table T4*: $T4[(y \ll 4)|(c_E \ll 2)|c_O] = \mathbf{te}[4y+c_E] \cdot \mathbf{to}[4y+c_O]$ — all 16 values of the level- $(d-2)$ node at y , which pairs level- $(d-1)$ positions y and $y + q_1$, i.e. the four leaves $y, y+q_1, y+q_2, y+q_1+q_2$, selected by four committed bits.

Each family costs $O(2^d)$ per set and is amortised 2^s -fold. The *T4* build collects an indexed **Vec**<[Gf; 16]> and flattens it in place: a parallel nested **flatten** treats every 16-entry row as its own parallel iterator and makes the collect unindexed — measured $\approx 836\times$ slower than the same arithmetic run serially (upstream fix **e19b0e1**).

Sizes. Everything the lazy prover holds, with the $n = 28$ instantiation ($d = 17, s = 11, W = 1$):

object	size	$n = 28$	live during
transposed bits	$2^n/8$ B	32 MiB	the whole forest
τ halves	$2^d \cdot 16$ B	2 MiB	the leaf layer
pair	$2^{d-1} \cdot 16$ B	1 MiB	the build
te , to	$2^{d+1} \cdot 16$ B	4 MiB	the pair layer
<i>T4</i>	$2^{d+2} \cdot 16$ B	8 MiB	build + JIT (L/4)
stored chain	$2^{n-2} \cdot 16$ B	1 GiB	layers $0..d-4$
JIT level $d-2$	$2^{n-2} \cdot 16$ B	1 GiB	layer $d-3$ only
LUT stashes	$2^{d+2} \cdot 16$ B	8 MiB	one bottom layer
first dense buffers	$2^{n-3} \cdot 16$ B	512 MiB	the bottom layers

The shared tables are all in the megabytes and the per-tree bits in the kilobytes; the only gigabyte-scale objects left are the stored chain and the JIT level, and §5.3 keeps those two from ever being live at once.

5.3 The level ladder

Layer ℓ 's phase A consumes level $\ell + 1$, so the prover must decide, for each of the d levels, how it comes into existence. There are three answers.

- **Stored** — built once bottom-up by `build_levels` and held level-major until its layer runs. Each tree's chain is built independently (parallel across trees) by repeated `parent_halves_top` — $v[i] = E[i] \cdot O[i]$, re-split at the new midpoint — then the chains are regrouped level-major and the roots fall out of level 1. `drive_grouped` takes each level with `mem::take` as its layer starts, so the chain shrinks as the forest climbs.
- **JIT** — regenerated from bits and tables at the start of its own layer, consumed by that layer's rounds, dropped.
- **Implicit** — never $\mathbb{K}$ -values at all: the layer's first one, two or three rounds run case-LUT bodies straight off the bits (§5.4), and the first dense buffers to appear are already folded ones.

The interesting level is $d - 2$. It is the highest level a ≤ 16 -case table still reaches (Identity 1) and the largest level whose layer could be driven from bits, and at 2^{n-2} values it is also the biggest thing the prover would ever store. The three schedules are precisely the three answers to “what happens to level $d - 2$ ”, and they differ in nothing else:

	L/2 (12)	L/4 (default)	L/8 (18)
chain tops at	level $d-2$	level $d-3$	level $d-4$
per position of the top	1 <i>T4</i> gather	2 gathers, 1 mul	4 gathers, 3 muls
level $d-2$	stored	JIT at layer $d-3$	implicit (T4Bits)
level $d-3$	stored	stored	JIT at layer $d-4$
layer $d-2$ (pair)		Pair3Bits over (te, to)	
layer $d-1$ (leaf)		Leaf3Bits over τ	
peak resident	$\approx 2^n \cdot 8 \text{ B}$	$\approx 2^n \cdot 4 \text{ B}$	$\approx 2^n \cdot 2 \text{ B}$
needs depth	≥ 4	≥ 4	≥ 5

The build. `gen_top` produces the chain's top level directly from the bits and *T4*, one subcube of Identity 1 per position: L/2 reads one *T4* entry (= level $d-2$), L/4 multiplies the pair $\{y, y + 2^{d-3}\}$ (= level $d-3$), L/8 the quadruple $\{y, y + 2^{d-4}, y + 2^{d-3}, y + 3 \cdot 2^{d-4}\}$ (= level $d-4$). Levels d , $d-1$ and (below L/2) $d-2$ therefore never exist at build time: each of their positions is gathered exactly once, with the materialise-then-read round trip removed. The chain that remains sums to $\sum_{\ell \leq \text{top}} 2^\ell \approx 2^{\text{top}+1}$ values per tree — a quarter of the leaf count under L/4, an eighth under L/8, whence the names.

The JIT layer. Under L/4, layer $d - 3$ regenerates level $d - 2$ per tree (2^{d-3} values per half, one *T4* gather each) into **Dense** buffers that die when the layer ends; under L/8, layer $d - 4$ does the same for level $d - 3$ at two gathers and a multiply per value. By default that generation pass is not just a write: `jit_layer_generate` accumulates the layer's opening round(s) *as it writes*, so the level is never re-read for a message pass it could have paid for during generation. With the driver's double-fold on (F2Z.EQF.DOUBLE, default) and $k \geq 3$, the pass accumulates the whole 3×3 bivariate grid (`dense_jit_fused_grid`): nine field elements per tree, from which the driver derives *both* round 1 and round 2's messages, so its first read of the level is round 3's pass — which then folds two deferred challenges at once. Otherwise the pass accumulates round 1's coefficient

triple only (`dense_jit_fused_round1`). Either way the result reaches the driver through its `pre_round1` hook and is value-exact against the message pass it replaces: the same weight-folds and wide products, XOR-accumulated (order-free) and reduced once, the reduction being $\mathbb{F}_2$ -linear. `F2Z_JIT_GRID=0` keeps the triple, `F2Z_JIT_R1=0` the plain write.

Shallow trees. The guards of §5.1 at $k = \ell$ give the fallback ladder directly, and the code takes exactly these branches:

depth d	what runs	the binding constraint
< 3	the eager prover over dense leaves	any LUT round needs $k \geq 2$
3	chain from level $d-1$, then <code>LeafBits</code>	<code>Pair2Bits</code> needs $k = d-2 \geq 2$
4	L/4 (forced), <code>Pair2Bits</code> + <code>Leaf2Bits</code>	<code>Leaf3Bits</code> needs $k = d-1 \geq 4$
≥ 5	the full ladder above	—

At depth 3 the chain’s top level is built by `build_column_layer1_halves`, a zero-multiply case copy that reads the $r = 1$ cases straight out of v and `pair` rather than through the driver. At depth 4 the L/8 request degenerates (its `T4Bits` layer would need $k = d - 3 \geq 2$) and falls back to L/4. `F2Z_LUT3=0` forces the shallower `Pair2Bits`/`Leaf2Bits` prefixes at every depth — the same code path the depth-4 edge takes.

Only the live columns are built. A committed column whose bits are all zero has every leaf $\alpha^0 = 1$, hence every level and its root = 1: a full 2^d -value pipeline (bit extraction, T4 gathers, JIT regeneration, one dense group per layer) to prove a constant. Since a phase-A round message is $\sum_c \text{eq}(c, z_c) H_c(\cdot)$ with H_c linear in the group’s scale, and every constant-1 tree has the *same* H_c , the trailing run of zero columns collapses into *one* synthetic group with all-ones buffers and scale $\sum_{c \geq \text{live}} \text{eq}(c, z_c)$ (`col_elide`, `live_cols`; the absorbed root vector is still padded with the known 1s, so the statement is unchanged, and on a JIT layer the synthetic group is born inside the generation pass, since the driver cannot recompute a fused round-1 entry itself). Char-2 addition is XOR and re-association is exact, so the transcript is byte-identical to the un-elided forest (`col_elision_matches_full`): a witness of N cells padded to 2^n pays the forest for $\lceil N/2^{t+\log_2 W} \rceil$ columns instead of 2^s . Everything in this section runs over the `live` leading trees only.

Remark 2 (Why the ladder stops at L/4). *Going down the ladder buys memory cheaply: L/8 costs a measured +5–13% of prove at the 2-column model shapes, and is what makes $n \geq 31$ possible on a 16 GB box (§6). Going up does not pay symmetrically. L/2 removes a whole T4 sweep over every tree — the JIT regeneration — but doubles the resident set, and measured across three shapes including a deep proxy at $4 \times$ L2 it comes out flat to worse than L/4. The reason is a useful rule of thumb for this prover: one T4 gather sweep costs about one streaming pass over the level it would have saved, so the choice of schedule is a near-pure memory decision with a near-zero time gradient. The time levers live inside the round bodies, not in the ladder.*

5.4 Inside the case-LUT rounds

Fix a layer with $k = \ell$ variables. Round j binds the low variable over 2^{k-j} slots; slot b owns entries $2b$ and $2b+1$ of each side and contributes to the three coefficients of §4,

$$A_0 = \sum_b w_b L[2b] R[2b], \quad A_2 = \sum_b w_b \Delta L_b \Delta R_b, \quad A_1 = \left(\sum_b w_b L[2b+1] R[2b+1] \right) + A_0 + A_2,$$

with $w_b = V_j[b]$ the suffix-tensor weight and $\Delta L_b = L[2b+1] + L[2b]$ (char 2, so $+$ $=$ $-$; the A_1 form is the char-2 reading of $A_1 = H(1) - A_0 - A_2$). A bit-driven round computes these sums *without the L, R arrays existing*: the entries are per-position case selects, so each slot’s three contributions are themselves case selects out of tables precombined over the same bits.

The leaf round: zero multiplications. At the leaf layer the entries are *defined* as $L[i] = 1 + m_{L,i} \tau_L[i]$ and $R[i] = 1 + m_{R,i} \tau_R[i]$, with m the committed bits (**lbits**, **rbits**) and τ the shared halves. Expanding one slot and writing $m_{L\varepsilon} = m_{L,2b+\varepsilon}$:

$$w_b L[2b] R[2b] = w_b + \underbrace{m_{L0} w_b \tau_{L0} + m_{R0} w_b \tau_{R0} + m_{L0} m_{R0} w_b \tau_{L0} \tau_{R0}}_{\mathbf{t_a0}[(b \ll 2) \mid m_{L0} \mid m_{R0} \ll 1]},$$

$$w_b \Delta L_b \Delta R_b = w_b (m_{L0} \tau_{L0} + m_{L1} \tau_{L1}) (m_{R0} \tau_{R0} + m_{R1} \tau_{R1}) = \sum_{\varepsilon, \delta \in \{0,1\}} m_{L\varepsilon} m_{R\delta} w_b \tau_{L\varepsilon} \tau_{R\delta},$$

the second line using the char-2 cancellation of the 1s inside ΔL and ΔR — this is where “bit-affine” pays. Both lines are subset sums of the *eight* products $w_b \tau_{L\varepsilon}$, $w_b \tau_{R\delta}$, $w_b \tau_{L\varepsilon} \tau_{R\delta}$ of that slot, and the leftover constants w_b sum to one per-set scalar (**w_sum**) added once at the end of the round. **build_leaf_tables** precombines the subsets into three per-position tables — **t_a0** and **t_a1** (4 cases each: the even and the odd entries) and the $\Delta\Delta$ table (16 cases) — for 8 multiplies and ≈ 20 additions per slot, once per τ set. A *tree*’s round-1 message then costs, per slot: one aligned 2-bit field per side (32 slots to a **u64**), three unconditional indexed loads, three additions. No multiplications, and no branch on a committed bit — a branchy skip-the-multiply-by-one variant was measured slower, 50/50 random bits making it mispredict-bound.

Two forms of the $\Delta\Delta$ table. The 16 $\Delta\Delta$ cases are subset sums of only four products, so the table has an interchangeable factored form (**LeafA2**, selected by **F2Z_LEAF_A2_FACTORED**, auto at $2^{k-1} \geq 2^{15}$): *precombined* spends $16 \cdot 2^{k-1}$ entries for one load per slot; *factored* keeps the four raw cross products ($4 \cdot 2^{k-1}$ entries, one sequential 64-byte line per slot) and selects them with four branchless masked adds that reduce pairwise, so the accumulator still takes one add. The trade is footprint against work: the precombined leaf tables are $384 \cdot 2^{k-1}$ bytes (12.6 MB at $k = 16$), and once they stop co-residing in the P-cluster L2 the factored form wins outright — at $n = 30$ the leaf round drops $2.1\times$ ($307 \rightarrow 147$ ms, prove -6.5%) and becomes far less run-to-run volatile, while at $n = 26$ (6.3 MB, still resident) it loses ≈ 1.2 ms. Both accumulate the identical field element (**leaf_a2_forms_agree** pins it case by case).

Folding without materialising, and the stash. Round 1’s fold must produce round 2’s entries. On a bit-affine side,

$$v' = v_0 + \rho(v_1 + v_0) = (1 + \rho)v_0 + \rho v_1 = 1 + m_0(1 + \rho)\tau_0 + m_1 \rho \tau_1,$$

since $(1 + \rho) + \rho = 1$: the folded entry is a 4-case function of the bit pair (m_0, m_1) — Identity 2 made concrete. **build_leaf_fold_tables** tabulates those four cases per position, so a *tree*’s fold is one indexed load per entry.

The deeper variants do not *execute* that fold; they *keep* it. Its table has exactly the layout of a level- $(d-1)$ case set (**Pair2TauSet**: 4 cases per position, two sides), so **Leaf2Bits** stashes it as round 2’s *value* tables and runs the 16-case product-layer body over them, materialising only at round 2’s fold. **Leaf3Bits** iterates once more: round 2’s fold tables (16 cases per position) become

round 3's value tables, read inline, and dense buffers appear only at round 3's fold. **Pair3Bits** does the same one level up. The economics are the point: materialising at round j 's fold costs 2^{k-j} values per side *per tree*, whereas the stash costs $4 \cdot 2^{k-j}$ entries (a 4-case set) or $16 \cdot 2^{k-j}$ (a 16-case one) per side *per τ set* — a factor *live*/4 resp. *live*/16, i.e. 512 $\times$ and 128 $\times$ at the deployed $s = 11$. It is the LUT prefix depth, not the ladder, that fixes the bottom layers' resident set.

The 16-case body. Four rounds share one body (**pair2_factored_body**): round 1 of **Pair2Bits** and of **Pair3Bits** over (**te**, **to**), and round 2 of **Leaf2Bits** and of **Leaf3Bits** over the stashed value sets. Slot b 's two entries per side are 4-case selects, so the slot is keyed by $(c_{E0}, c_{E1}, c_{O0}, c_{O1})$: the accumulators want the weighted product $w \text{TE}[c_{E0}] \text{TO}[c_{O0}]$ at the even position, its odd twin, and the product of the two differences. Two forms compute the same field elements:

- *precombined* (**F2Z_PAIR2_FACTORED=0**): four 16-case tables per slot (**t_a0**, **t_a1**, the weighted ΔTE , the raw ΔTO), so a slot is three loads and *one* wide multiply — but the four tables hold $4 \cdot 16 \cdot 2^{k-1} = 2^{k+5}$ entries, 2^{k+9} bytes, 16 MiB at the $n = 28$ pair layer ($k = 15$), and the four gather streams' misses dominate the round;
- *factored* (the default): only the suffix-weighted $w\text{te}[i] = V_1[i \gg 3] \cdot \text{te}[i]$ is built, and the body recombines against the set's raw **to** — three wide multiplies per slot against a streamed set of $2 \cdot 4 \cdot 2^k$ entries, 2^{k+7} bytes: 4 $\times$ smaller, and L2-resident where the other is not.

The two agree exactly because distributivity is exact: $w\text{TE}_0 + w\text{TE}_1$ is the precombined $w(\text{TE}_0 + \text{TE}_1)$, the products keep the build's association $(w \cdot \text{TE}) \cdot \text{TO}$, and the deferred reduction is $\mathbb{F}_2$ -linear. The lesson generalises to the inline rounds below: multiplies are cheap next to bytes that miss.

Reading the cases. Each round down the cascade reads a wider aligned bit window per slot, and never a straddling one (the E/O offset 2^k is even at every round, so the O-side window stays inside its word too):

round body	windows per slot	helper
leaf round 1	two 2-bit fields at $2b$	inline
Leaf2/Leaf3 round 2	two nibbles at $4b$	leaf2_cases
Leaf3 round 3	two bytes at $8b$	leaf3_idx
Pair2/Pair3 round 1	four 2-bit fields at $2b, 2b+2^k$	pair2_cases
Pair3 round 2	four nibbles at $4b, 4b+2^k$	pair3_idx
T4Bits round 1	16 single bits (4 entries $\times$ 4 leaves)	t4bits_idx

The stashed tables stay keyed by the *original* committed bits throughout — a **Leaf3Bits** round-3 entry is $f[(p \ll 4)|(c_0 \ll 2)|c_1]$ over the two adjacent original bit pairs of positions $4p \dots 4p+3$, and **Pair3Bits**' round-2 key interleaves the **lbits/rbits** pairs the same way — which is why nothing per-tree is written between rounds. The inline rounds (**Leaf3** round 3, **Pair3** round 2, **T4Bits** round 1) then just run the ordinary dense single-pair body over four gathered values: no precombined slot tables, because at that width the table would cost more bytes than the multiplies it saves.

The 4-leaf round (T4Bits, L/8 only). One level further up, an entry is a single 16-case select from the shared **T4** — one gather per value, read inline in round 1, dense buffers at its fold. The layer's input level is neither stored nor regenerated; combined with the JIT layer below it, this is what makes L/8 an eighth.

Prefetch. Every LUT gather is data-dependent (committed bits pick the line), which defeats the hardware prefetcher, but the indices are cheaply recomputable ahead from the sequential bit words. Both the stash-gather rounds (F2Z_LUT_PRFM: Pair3 round 2, Leaf3 round 3, and their two materialising folds) and the $T4$ build/JIT sites (F2Z.T4_PRFM) issue `prfm pldl1keep` 16 slots ahead, size-gated to the DRAM-scale shapes — they *lose* at L2-resident sizes, where the index recompute and the extra load-store pressure beat an L2 hit. Measured at $n = 30$: the four stash scopes drop 20–25 % for –4.8 % prove, and the $T4$ sites –5.3 %; at $n \leq 28$ both are off by default. Semantically inert.

Where the time shows up. The prover’s profiling scopes name every stage of this section individually, so a run attributes to the ladder directly (OBLONG_PROFILE=1 prints the phase split; `examples/prof_probe.rs` dumps the whole tree):

stage	scopes
build, bit/JIT generation	<code>mf:build_levels</code> , <code>mf:bitgen</code>
layer phases	<code>mf:phaseA</code> , <code>mf:phaseB</code>
shared tables	<code>eqf:suffix</code> , <code>eqf:leaf_tables</code> , <code>eqf:pair2_tables</code> , <code>eqf:leaf2_tables</code>
LUT round messages	<code>eqf:msg: leaf_r1</code> , <code>leaf2_r2</code> , <code>leaf3_r2</code> , <code>leaf3_r3</code> , <code>pair2</code> , <code>pair3_r1</code> , <code>pair3_r2</code> , <code>t4bits</code>
dense round messages	<code>eqf:msg:dense_r1</code> , <code>eqf:msg:dense_postlut</code> , <code>eqf:grid</code> , <code>eqf:fmsg</code> , <code>eqf:fmsg2</code>
folds	<code>eqf:fold: stash</code> , <code>leafmat</code> , <code>leaf2mat</code> , <code>leaf3mat</code> , <code>pair2mat</code> , <code>pair3mat</code> , <code>t4mat</code> , <code>dense</code>

5.5 Why byte-identity holds, and how it is pinned

Every device above is a re-*association* of the same field arithmetic, never a different computation. Four principles cover all of them:

1. *Precombination is distributivity.* A case-table entry is a subset sum (or a product of subset sums) of terms the dense path would compute per entry; field addition and multiplication are exact and associative, so the accumulated sums are the same elements.
2. *Char-2 cancellation is an identity.* $1 + 1 = 0$ removes the leaves’ 1s from every difference, and $(1 + \rho) + \rho = 1$ keeps a folded bit-affine entry affine; these are the rewritings that make the leaf round multiply-free and the fold tables 4-case.
3. *Wide accumulation reduces to the same value.* Deferring the polynomial reduction to one per accumulator per round is exact because the reduction is $\mathbb{F}_2$ -linear — and, for the same reason, accumulation order does not matter, which is what licenses the fused JIT generation passes.
4. *Fusion reorders passes, not values.* Deferred folds, the fused fold+message pass, the double-fold grid and the JIT pre-round hooks all emit the same field elements in the same transcript order; only the physical traffic changes.

Consequently the lazy prover does not merely “verify the same” — it emits the *same transcript*, byte for byte, so the verifier and the proof codec never learn which schedule ran. `lazy_matches_eager` pins this against the eager prover across depths 3–9 (covering the eager fallback, the depth-4 degenerate JIT edge, the scalar and word-wise $T4$ generation paths at $q_1 < 64$, $= 64$, $= 128$), $W \in \{1, 4\}$, and *every* schedule (L/2, L/4, L/8): roots, every layer’s phase-A and phase-B

messages, every closing pair, the exit claim, and a challenge drawn *after* the run to compare the final transcript states. `lazy_multi_matches_eager` does the same for the batched forest (§8) and `col_elision_matches_full` for the elided one. This is what makes the knob inventory of Table 1 safe: every knob changes scheduling, never semantics.

Knob	Default	Effect (always byte-identical)
<code>F2_FOREST_SCHEDULE=18</code>	off (L/4)	L/8 memory schedule (§5.3)
<code>F2_FOREST_SCHEDULE=12</code>	off (L/4)	L/2: level $d-2$ stored, no JIT
<code>F2Z_LUT3=0</code>	on	shallower <code>Pair2/Leaf2</code> LUT prefixes
<code>F2Z_COL_ELIDE=0</code>	on	build the all-zero column tail too
<code>F2Z_EQF_FUSE=0</code>	on	eager two-pass fold (no pass fusion)
<code>F2Z_EQF_DOUBLE=0</code>	on	one variable per dense pass (no double-fold)
<code>F2Z_JIT_R1=0</code>	on	unfused JIT round-1 message pass
<code>F2Z_JIT_GRID=0</code>	on	JIT fuses round 1 only, not the grid
<code>F2Z_LEAF_A2_FACTORED=0/1</code>	auto ($\geq 2^{15}$)	$\Delta\Delta$ table form, leaf round
<code>F2Z_PAIR2_FACTORED=0</code>	on	precombined 16-case pair tables
<code>F2Z_LUT_PRFM=0/1</code>	auto ($\geq 2^{14}$)	stash-gather software prefetch
<code>F2Z_T4_PRFM=0/1</code>	auto (≥ 16 MiB)	T4 build/JIT software prefetch
<code>F2Z_EQF_NOKERNEL=1</code>	off	bypass fused NEON kernels (diagnostic)

Table 1: The forest’s scheduling knobs. Each is read once per prove/process; none changes the proof bytes.

6 Memory and cost

Memory model. With 16-byte $\mathbb{K}$ -elements and $W = 1$ (so $2^{d+s} = 2^n$ leaves), the resident forest set is capped by the largest stage of the ladder:

$$\text{L/4: } \approx 2^{n-2} \cdot 16 \text{ B} = 2^n \cdot 4 \text{ B}, \quad \text{L/8: } \approx 2^{n-3} \cdot 16 \text{ B} = 2^n \cdot 2 \text{ B},$$

(the stored chain and the JIT level are the same size and not simultaneously live: the chain is consumed front-first, `mem:take` per layer, before the JIT layer runs). On top sit the commit hint ($\approx 2^n \cdot 0.7 \text{ B}$: packed rows/columns, codeword, Merkle tree), the transposed bit halves ($2^n/8 \text{ B}$), and the shared tables ($O(2^d)$ per set; the largest, T4, is $16 \cdot 2^{d-2}$ entries — 8 MiB at $d = 17$). The model matches the measured peaks below to within $\approx 5\%$; it is why the 16 GB ceiling sits at $n = 32$ (under 18) and why $n \geq 31$ requires 18 at all.

Measured (Apple M4 16 GB, LTO + unchecked, fusion defaults, rate-1/2 $k=4$ configs, medians; 2026-07-18). The `forest+presum` column is the phase this note describes (the α -power tables, the forest proper, and the pre-sumcheck of §7); its share of prove rises from 81 % to $\approx 96\%$.

n	(t, s)	forest+presum	lig. open	prove	verify	peak	sched
16	(10, 6)	3.25 ms	0.72 ms	4.03 ms	1.21 ms	0.69 MB	L/4
20	(13, 7)	9.13 ms	1.62 ms	10.9 ms	1.83 ms	7.7 MB	L/4
24	(15, 9)	42.5 ms	21.5 ms	62.1 ms	2.48 ms	85.0 MB	L/4
26	(16, 10)	131 ms	25.3 ms	159 ms	3.14 ms	325 MB	L/4
28	(17, 11)	530 ms	53.6 ms	579 ms	4.41 ms	1.27 GB	L/4
30 [†]	(18, 12)	2.98 s	139 ms	3.15 s	5.55 ms	4.95 GB	L/4
32 [†]	(19, 13)	$\sim 96\%$	$\sim 4\%$	29.8 s	11.8 ms	11.5 GB	18

([†]: the older $k=6$ generation, pending a memory-fresh re-measure; full tables and the measurement protocol in `README.md`.) Proof bytes stay forest-light: forest-side 7.7 KiB ($n=16$) $\rightarrow$ 50.7 KiB ($n=28$) $\rightarrow$ 151.8 KiB ($n=32$), of which the sent folds are $16 \cdot 2^s$ B (§2); the Ligerito blob dominates the stream throughout. Threading: the build is parallel across trees, round bodies across groups (with a minimum batch so tiny late rounds amortise dispatch), phase B and the upper layers effectively serial; the MT/ST prove ratio grows from $\approx 1.6\times$ at $n = 22$ to $\approx 3.6\times$ at $n = 28$. All deployed shapes run a *single* chunk (with $q = 2^{100}-15$, $c_w = 126 - t \geq 100$ for $t \leq 26$); chunking multiplies the forest count only at $t + W > 27$.

7 The exit claim and its discharge

The forest ends holding $e_d \approx \tilde{\Lambda}(z_x, z_c)$ — still entangled with the α -power weights. Two further steps (`prove_int_eval_merged_common`) turn it into something the opener understands; they are the “presum” half of the `forest+presum` phase.

From the exit claim to a bit-MLE claim. By (1) and multilinearity (the all-ones table extends to 1):

$$e_d - 1 = \sum_{i,c} \text{eq}(i, z_x) \text{eq}(c, z_c) M[c][i] \tau(i) = \sum_{i \in \{0,1\}^d} q_{\text{rowbit}}[i] \cdot m[i],$$

with $q_{\text{rowbit}}[i] = \text{eq}(i, z_x) \tau(i)$ (`row_bit_weights` at $\rho = z_x$; its $2^t \approx q_{\text{bits}}$ -bit exponentiations go through the same fixed-base comb) and $m[i] = \sum_c \text{eq}(c, z_c) M[c][i]$ the z_c -combined row (`xi_combined_rows`). A d -round *degree-2* sumcheck over this product (`MultiDegreeSumcheck`, one group) binds $i \rightarrow r^*$; dividing the final expected evaluation by the public $\widetilde{q_{\text{rowbit}}}(r^*)$ leaves $\mu = \widetilde{M}(r^*, z_c)$ — a pure bit-MLE evaluation at $(r^* || z_c)$, every α -dependent factor stripped to the verifier’s side. From here the ring switch sends the 128 partial evaluations s_v , the L chunks’ residuals are η -combined, and one recursive Ligerito call certifies the batch against the commitment — verifier note §6, out of scope here.

The chunked flow. `prove_mle_eval_mod_q_ligerito` runs the full forest + presum *per chunk*, sequentially on one transcript (chunk l ’s α -power tables are built and dropped inside its run), then does all ring-switch messages, then the single Ligerito call. Per chunk the proof stream carries: the `MergedForestProof`, the folds $u^{(l)}$, the pre-sumcheck, and s_v (`proof_codec.rs`).

8 Variants: the batched forest and the RLC families

Multi-claim batching (`prove_merged_forest_lazy_multi`). N same-shape claims — each 2^s trees over its own bits and τ chains — run as *one* forest of $N \cdot 2^s$ trees, tree index $(\nu \ll s) | c$ with the claim index high, sharing every layer’s rounds and messages. Per-claim table selection rides the driver’s `tau_set` index (the kernels are untouched); table sets are deduplicated by τ -chain identity, so same-point claims share one set. N must be a power of two; callers pad with zero-weight dummies, whose τ chains are all 1 and whose leaves are therefore identically 1 regardless of the bits they carry. L/4 and L/8 are both honoured (L/2 is a single-prover schedule; the batched path reads the knob as L/4-or-L/8); pinned byte-identical to the eager prover over the concatenated leaf tables, dummy arm included.

RLC claim families (experimental). The mod- q RLC families (`docs/rlc-family-note/`) replace the bit-affine leaf with a 2^j -case select on j committed column bits: $\Lambda[c][i] = \text{case_pow}[i][m(i, c)]$, case $0 = \alpha^0 = 1$. For $j = 2$ (`prove_merged_forest_lazy_rlc2`) this is a pure *rewiring* of the machinery above, one level shifted: the leaf layer is structurally a product layer (`Pair3Bits` over `te[4y+m] = case_pow[y][m]`), level $d-1$ is a `T4Bits` round over `te · to` products, level $d-2$ is Dense JIT from paired gathers, and the chain tops at $d-3$ — the L/4 shape with untouched kernels, hence byte-identical to the eager forest over the same leaves (pinned). For $j = 3, 4$ (`prove_merged_forest_lazy_rlc_general`) no 2^j -case leaf kernels exist yet, so the bottom layers run Dense with JIT-generated buffers gathered from a shared 2^{2j} -case top-pair product table $T2$; dedicated 8/16-case leaf-round kernels are the stated open lever. (A Dense-JIT arm for the *bit-affine* $j \geq 3$ forests was measured slower than the eager path and closed.)

9 What to trust, and what this note is not

The prover described here is exercised by: the both-schedule byte-identity pins (`lazy_matches_eager`, `lazy_multi_matches_eager` — roots, every message, exit claim, transcript state, across the depth-regime and W matrix), the GKR roundtrip and tamper rejections (`merged_forest_roundtrips`), the mod- q end-to-end roundtrips with range/generator/tamper arms, and the NEON/scalar field-pipeline equivalence (`neon_mul_matches_scalar_pipeline`). Soundness of what the forest *binds* — the chain from the absorbed roots through the layer sumchecks to the exit claim, the exponent injectivity, and the oracle handoff — is argued in the verifier note and `docs/DESIGN.md`; this note’s claim is narrower and mechanical: the lazy, merged, fused prover computes *the same protocol* as the eager reference, at $O(\ell + s)$ payload per layer, $\approx 2^n \cdot 4\text{B}$ (or $2^n \cdot 2\text{B}$) of working set, and 81–96 % of an F2Z prove. The standard caveats carry over unchanged: flock’s Merkle domain separation is a flagged upstream item, the ad-hoc Ligerito config regime ($m < 22$) is unaudited, and the scheme is not zero-knowledge.

References

- [1] S. Goldwasser, Y. T. Kalai, G. N. Rothblum. *Delegating computation: interactive proofs for muggles*. J. ACM 62(4), 2015 (STOC 2008).
- [2] J. Thaler. *Time-optimal interactive proofs for circuit evaluation*. CRYPTO 2013. (The product-tree GKR specialisation the forest layers instantiate.)
- [3] T. Xie, J. Zhang, Y. Zhang, C. Papamanthou, D. Song. *Libra: succinct zero-knowledge proofs with optimal prover computation*. CRYPTO 2019. (The eq-factored linear-time GKR prover lineage.)
- [4] A. Gruen. *Some improvements for the PIOP for ZeroCheck*. IACR ePrint 2024/108. (Tail-only round messages via the claimed-sum reconstruction.)
- [5] L. Soukhanov. *The characteristic-2 field-switching note*, §9. (The integer-MLE-over- $\mathbb{F}_2$ construction F2Z realises; internal.)